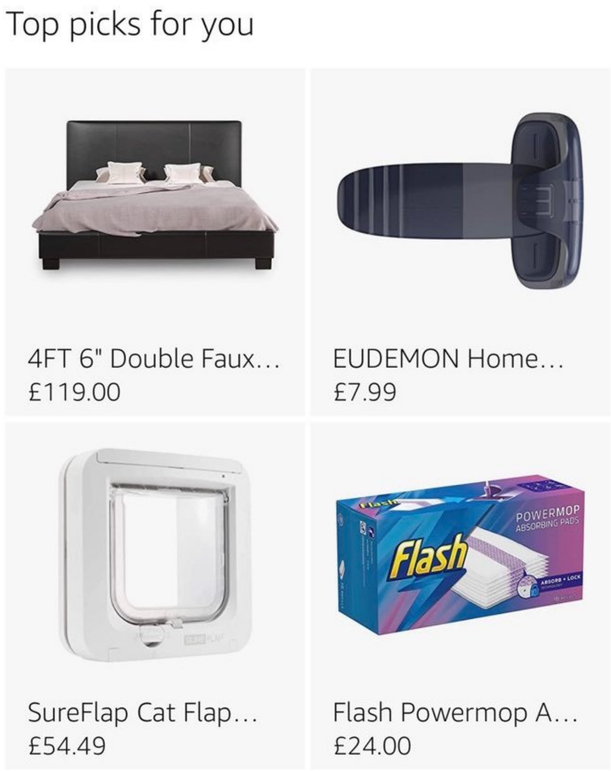
CS430/910: Foundations of Data Analytics

Recommender Systems 1 | Ayşe Sunar

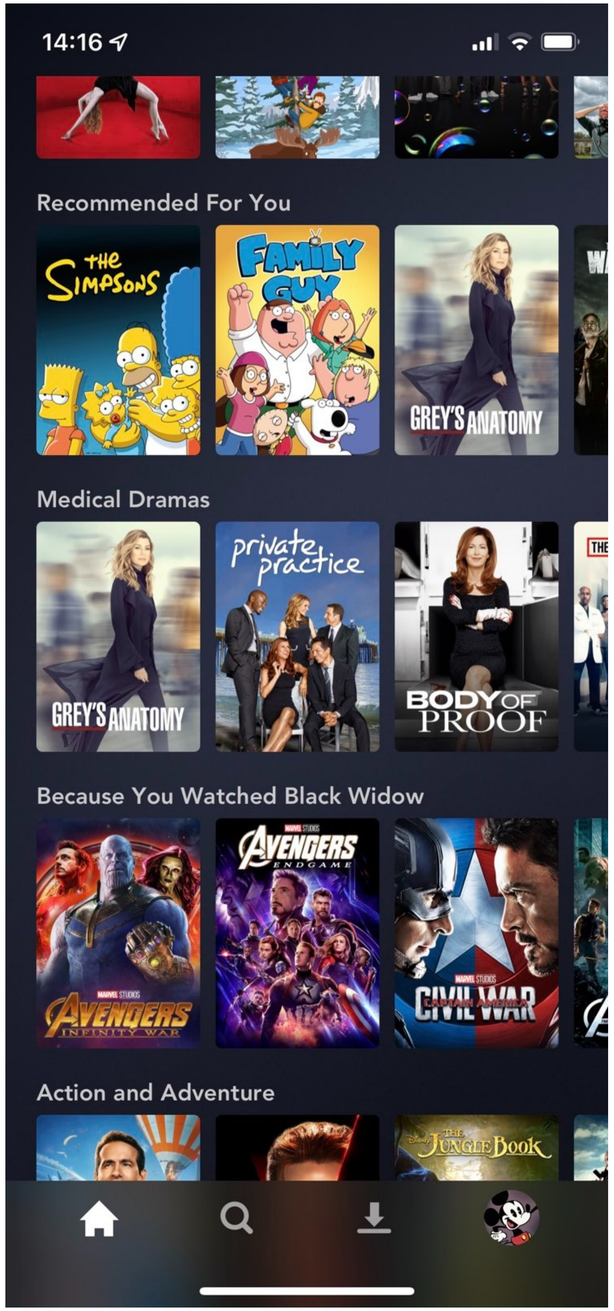
Recommendations

- A modern problem: a very large number of possible items
  - Which item should I try next, based on my preferences?
- Arises in many different places:
  - Recommendations of content: books, music, movies, videos...
  - Recommendations of places to travel, hotels, restaurants
  - Recommendations of food to eat, sites to visit
  - Recommendations of articles to read: news, research, gossip



Recommendations

- Each person has different preferences/interests
  - How to elicit and model these preferences?
  - How to customise recommendations to a particular individual?



Recommender Systems

- Recommender Systems: produce tailored recommendations
  - Inputs: ratings of items by users
    - Possibly also: user profiles, item profiles
  - Outputs: for a given user, output a list of recommended items
    - Or, for a given $(user, item)$ pair, output a predicted rating

Recommender Systems

- Ratings can be in many forms:
  - *Continuous Ratings* - on a continuous scale, e.g. a real number between 0 and 564.
  - *Ordinal Ratings* - Uses ordered categorical values, e.g. “Strongly Disagree”, “Disagree”, “Neutral”, “Agree”, and “Strongly Agree”.
  - *Binary Rating* - E.g. yes or no, thumbs-up or thumbs-down.
  - *Unary Rating* - Can only specify a positive preference for something, but not a negative preference (e.g. Facebook Like).

Ratings Matrix

- $m \times n$ matrix of ratings R, where r_{ij}, is rating of user i for item j
- Typically, matrix is large and sparse
  - Thousands of users (m) and thousands of items (n)
  - Each user has rated only a few items
  - Each item is rated by at most a small fraction of users
- If an entry (ij) is predicted rather than observed, it is denoted $\hat{r}_{ij}$ (with a hat/circumflex).
- Goal is to provide predictions $\hat{r}_{ij}$ for certain (user, item) pairs

	Item 1	Item 2	Item 3	Item 4	Item 5	...
User 1	5	3		1	2	
User 2		2		?		
User 3			5			
User 4	3	4		2	1	
User 5		?			4	
...						

Operational and Technical Goals

1. Relevance:
   - Recommend items that are relevant to a user's interests. Users are more likely to consume items that they find interesting
2. Novelty:
   - No point recommending items which the user has already seen in the past
3. Diversity:
   - Could suggest a list of the top-k items. However, if these items are too similar, users may not like any of the items. When Recommendation Systems recommend a list of items of different types, the user might like at least one of these items.

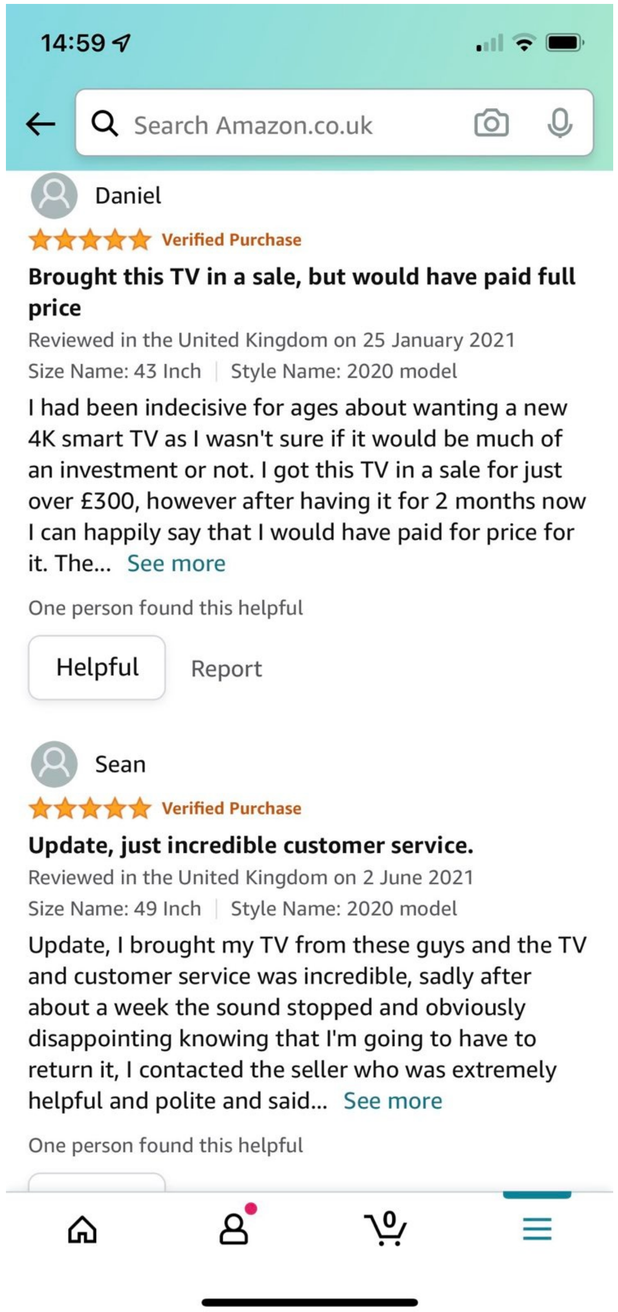
Operational and Technical Goals

4. Serendipity:
   - Items should be unexpected but still relevant. A user may consume one type of item, but there may be some latent interest in something else…
   - E.g. A new module in Computer Science is launched. Recommending this to you as CS430/CS910 students would be novel (you haven't studied the new module before as it didn't exist!), but it is not necessarily "serendipitous".
     - However, you may be recommended a module in Chemistry/Physics/ Medical School which may capture your interest.
     - Problem - More likely to recommend irrelevant items

Case Study

Amazon

- One of the pioneers of Recommendation Systems.
- Began in the 1990's as a book e-retailer, but now stocks virtually all sorts of products (books, CDs, Blu-rays, software, homeware, etc.).
- Amazon recommendations are based on provided ratings (explicit ratings) and by buying behaviour and browsing behaviour (implicit ratings).
- Ratings on a scale of 1-5 (1 = lowest, 5 = highest).

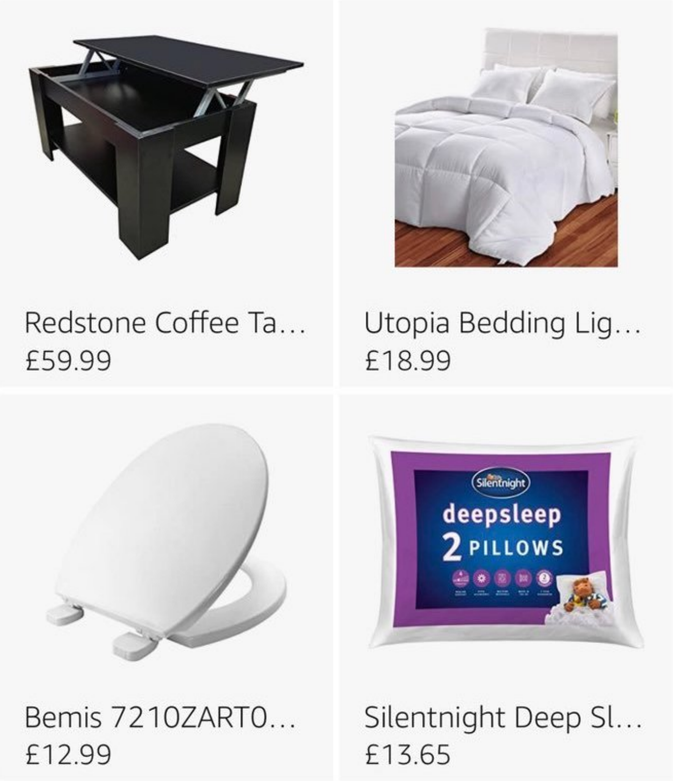


Case Study

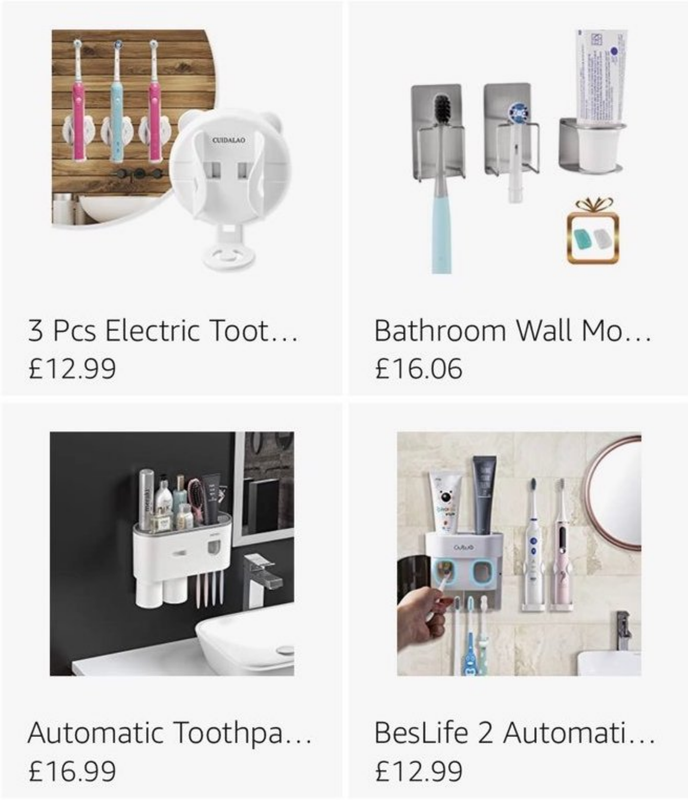
Amazon

Explanations for recommendations provided

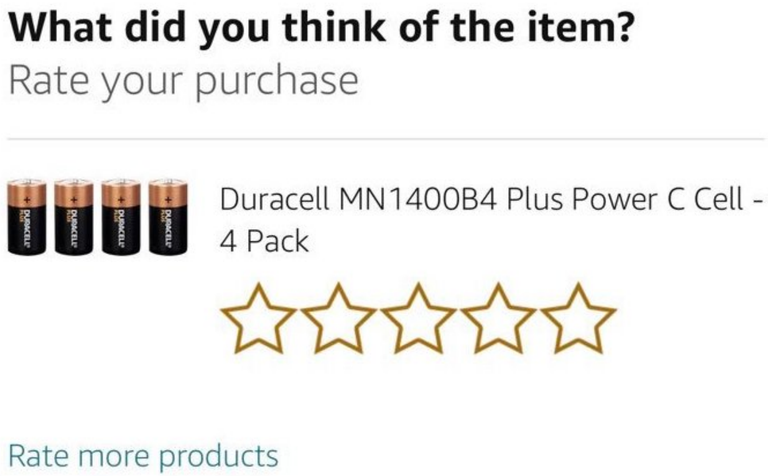
Inspired by your shopping trends



Related to items you've viewed



Ratings requested



All three screenshots taken from Amazon's iOS app home page.

Case Study

The Netflix Prize

- The Netflix Prize Contest was designed to compare various *collaborative filtering algorithms* contributed by contestants.
- Goal: Predict ratings of particular user-item combinations.
- Participants needed to predict ratings, which were scored by judges (or an equivalent automated system), with scores being added to a live leaderboard.
- Prizes were awarded based on improvement of Netflix's own recommendation algorithm, known as Cinematch, or by improvement of the best previous score by a certain threshold. https://www.youtube.com/watch?v=ImpV70uLxyw
- The Netflix prize has contributed several times to recommendation research[1,2].

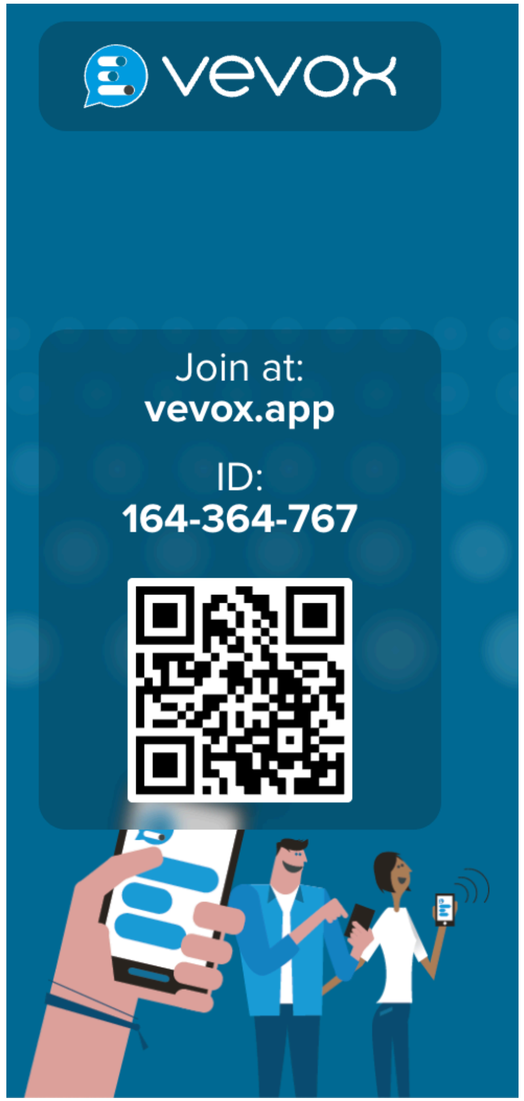
[1]Bell, R., Koren, Y. and Volinsky, C., 2007, August. Modeling relationships at multiple scales to improve accuracy of large recommender systems. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining* (pp. 95-104).
[2]Lohr, S., 2009. A $1 million research bargain for Netflix, and maybe a model for others. *The New York Times*, *22*.

Basic Models of Recommender Systems

- The basic models for Recommender Systems work with two kinds of data:
  1. User-Item Interaction (e.g. ratings, or buying behaviour)
  2. Attribute Information about the user or item (e.g. textual profiles, relevant keywords)
- Methods that use #1 are known as *Collaborative Filtering* methods, whereas methods that use #2 are known as *Content-Based Recommender* methods.
- It's not a case of one or the other. *Hybrid* systems can combine the various strengths of various types of Recommender Systems.

Good example of Recommender Systems

Your personal experience



Collaborative Filtering Models

- Uses the collaborative power of user ratings to make recommendations.
- Problem: Data is sparse.
  - Consider movie ratings - most users of a system would have only rated a tiny subset of movies.
- Ratings provided by the user are often called *observed ratings*, and ratings which are not provided are often called *unobserved ratings*.
- Basic idea of Collaborative Filtering: Unobserved ratings can be imputed because observed ratings are highly correlated across various users.

Collaborative Filtering Models

Example

- Consider two users, Alice and Bob, who have very similar tastes.
- If the observed ratings between both users are similar, then it is likely that the ratings in which only either Alice or Bob have specified a rating, would also be similar.
- This similarity can be used to make inferences about unobserved values.
- In the example to the right, we can use Alice's ratings to predict Bob's rating of *Toy Story*.

	Toy Story	Shrek	Iron Man	Detective Pikachu	Sonic the Hedgehog
Alice	4	2		1	3
Bob	?	2		1	2

Collaborative Filtering Models

- Collaborative Filtering methods can mostly be divided into two categories:
  1. Memory-Based Methods (or Neighbourhood-Based Collaborative Filtering)
     1. User-Based Collaborative Filtering
     2. Item-Based Collaborative Filtering
  2. Model-Based Methods

User-Based Collaborative Filtering

- Ratings provided by users similar to a target user A, are used to make recommendations for A.
- Predicted ratings of A are computed as the weighted average of ratings from these similar users, for each item.
- Memory-Based often called Neighbourhood-based: Neighbourhood refers to similar users or items.

User-Based Collaborative Filtering

- In order to determine the neighbourhood of target user i, the similarity to all other users is computed.
- We need a similarity function. Let's give an example.
- For the $m \times n$ ratings matrix R with m users and n items, let I_u denote the set of item indices for which ratings have been specified by the user (row) u.
  - E.g. if we have observed the ratings of items 1, 2 and 4 for user u, then $I_u = \{1,2,4\}$.
  - The set of items rated by both users u and v is given by $I_u \cap I_v$. Because of sparsity it is common for this to be empty.

User-Based Collaborative Filtering

- One possible similarity metric is the Pearson Correlation Coefficient, which is computed on $I_u \cap I_v$.
- First step is to compute the mean rating μ_a for each user a, using the observed ratings:

$$\mu_a = \frac{\sum_{k \in I_a} r_{ak}}{|I_a|}, \forall a \in \{1 \ldots m\}$$

User-Based Collaborative Filtering

- Then, the Pearson Correlation Coefficient between the rows (users) u and v is defined as:

$$\text{Sim}(u, v) = \text{Pearson}(u, v) = \frac{\sum_{k \in I_u \cap I_v} (r_{uk} - \mu_u) \cdot (r_{vk} - \mu_v)}{\sqrt{\sum_{k \in I_u \cap I_v} (r_{uk} - \mu_u)^2} \cdot \sqrt{\sum_{k \in I_u \cap I_v} (r_{vk} - \mu_v)^2}}$$

- Strictly speaking μ_u and μ_v should be computed only over the items rated by both users u and v, unlike in the computation of μ on the previous slide. However, such an approach is more computationally demanding and susceptible to variance.

User-Based Collaborative Filtering

Defining the Neighbourhood

- How do we define the user neighbourhood of a target user u?
  - One way: Use the set of k users with the highest Pearson Coefficient with the target, u.
    - Problem: The number of observed ratings in the top-k neighbourhood may vary significantly for each item.

	Item 1	Item 2	Item 3	Item 4	Item 5	...
User 1	?	?	3	5	1	
User 2	5	2	1	5	5	
User 3	4		4	5	2	
User 4	5		2	5	1	
User 5	4		2	5	1	
...						

User-Based Collaborative Filtering

Defining the Neighbourhood

- Possible solution - find the closest k users to the target user u separately for each item, such that each of these k users have an observed rating for that item.
  - The weighted average of these ratings can be returned as the predicted rating for that item.
- Question: What could we use to weigh each rating of users in the top-k neighbourhood?

	Item 1
User 1	?
User 2	4
User 3	5
User 4	4
User 5	4
...	

	Item 2
User 1	?
User 19	1
User 56	1
User 129	1
User 672	2
...	

	Item 3
User 1	?
User 28	3
User 55	4
User 494	4
User 913	2
...	

User-Based Collaborative Filtering

Defining the Neighbourhood

- Possible solution - find the closest k users to the target user u separately for each item, such that each of these k users have an observed rating for that item.
  - The weighted average of these ratings can be returned as the predicted rating for that item.
- Question: What could we use to weigh each rating of users in the top-k neighbourhood?
  - Answer: The Pearson Correlation coefficient of the owner to the target user, u.

	Item 1
User 1	?
User 2	4
User 3	5
User 4	4
User 5	4
...	

	Item 2
User 1	?
User 19	1
User 56	1
User 129	1
User 672	2
...	

	Item 3
User 1	?
User 28	3
User 55	4
User 494	4
User 913	2
...	

User-Based Collaborative Filtering

Scaling

- Some users may be overly generous (User 2?) with their ratings, whereas some may be overly cruel (User 4?)!
- The raw ratings must be mean-centred in a row-wise fashion.
- The *mean-centred rating* s_{uj} of a user u for an item j is defined by subtracting their mean rating from the raw rating r_{uj}:

$$s_{uj} = r_{uj} - \mu_u, \forall u \in \{1...m\}$$

	Item 1	Item 2	Item 3	Item 4	Item 5	...
User 1	5	3		4	4	
User 2	4	5	5	5	5	
User 3	5	4	3		1	
User 4	1		1	1	2	
User 5	3	1	2	4	1	
...						

User-Based Collaborative Filtering

Scaling

- The *mean-centred rating* s_{uj} of a user u for an item j is defined by subtracting their mean rating from the raw rating r_{uj}:

$$s_{uj} = r_{uj} - \mu_u, \forall u \in \{1...m\}$$

- Using these mean-centred ratings, would result in a mean-centred prediction. We can therefore add the mean rating of the target user u (μ_u) to the mean-centred prediction, to calculate a *raw rating prediction*.

User-Based Collaborative Filtering

Prediction Function

- Putting it together, the overall *neighbourhood-based prediction function* is:

$$\hat{r}_{uj} = \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot s_{vj}}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|} = \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot (r_{vj} - \mu_v)}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|}$$

where $P_u(j)$ is the set of k-closest users to the target user u who have observed ratings for item j.

User-Based Collaborative Filtering

Prediction Function

- Putting it all together, the overall *neighbourhood-based prediction function* is:

$$\hat{r}_{uj} = \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot s_{vj}}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|} = \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot (r_{vj} - \mu_v)}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|}$$

where $P_u(j)$ is the set of k-closest users to the target user u who have observed ratings for item j.

μ_u for converting mean-centred rating to raw rating.

User-Based Collaborative Filtering

Example

- Let the target user be user 3.
- We want to predict $\hat{r}_{31}$ and $\hat{r}_{36}$.
- First step, compute similarity between user 3 and all other users. Recall Pearson Correlation Coefficient:
- $$\frac{\sum_{k \in I_u \cap I_v} (r_{uk} - \mu_u) \cdot (r_{vk} - \mu_v)}{\sqrt{\sum_{k \in I_u \cap I_v} (r_{uk} - \mu_u)^2} \cdot \sqrt{\sum_{k \in I_u \cap I_v} (r_{vk} - \mu_v)^2}}$$

Item-Id ⇒ User-Id ⇓	1	2	3	4	5	6	Mean Rating	Cosine(i, 3) (user-user)	Pearson(i, 3) (user-user)
1	7	6	7	4	5	4	5.5	0.956	0.894
2	6	7	?	4	3	4	4.8	0.981	0.939
3	?	3	3	1	1	?	2	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2	0.645	-0.817

User-Based Collaborative Filtering

Example (Ignoring Mean-Centred Ratings)

Pearson(1,3) =

$$\frac{(6-5.5)*(3-2)+(7-5.5)*(3-2)+(4-5.5)*(1-2)+(5-5.5)*(1-2)}{\sqrt{0.5^2+1.5^2+(-1.5)^2+(-0.5)^2}\cdot\sqrt{1^2+1^2+(-1)^2+(-1)^2}}$$

$= 0.894$, (Pearson(2,3) = 0.939)

- Then, using the Prediction Function:
- $$\hat{r}_{uj} = \frac{\sum_{v \in P_u(j)} \text{Sim}(u,v) \cdot r_{vj}}{\sum_{v \in P_u(j)} |\text{Sim}(u,v)|}$$

Item-Id ⇒ User-Id ⇓⇓	1	2	3	4	5	6	Mean Rating	Cosine(i, 3) (user-user)	Pearson(i, 3) (user-user)
1	7	6	7	4	5	4	5.5	0.956	0.894
2	6	7	?	4	3	4	4.8	0.981	0.939
3	?	3	3	1	1	?	2	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2	0.645	-0.817

$$\frac{\sum_{k \in I_u \cap I_v} (r_{uk}-\mu_u)\cdot(r_{vk}-\mu_v)}{\sqrt{\sum_{k \in I_u \cap I_v} (r_{uk}-\mu_u)^2}\cdot\sqrt{\sum_{k \in I_u \cap I_v} (r_{vk}-\mu_v)^2}}$$

User-Based Collaborative Filtering

Example (Ignoring Mean-Centred Ratings)

- Then, using the Prediction Function:
- $\hat{r}_{uj} = \dfrac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot r_{vj}}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|}$
- $\hat{r}_{31} = \dfrac{7 * 0.894 + 6 * 0.939}{0.894 + 0.939} \approx 6.49$
- $\hat{r}_{36} = \dfrac{4 * 0.894 + 4 * 0.939}{0.894 + 0.939} = 4$
- So item 1 “should” be prioritised over item 6.
- User 3 “should” prefer items 1 and 6 over anything they’ve rated.

Item-Id ⇒ User-Id ⇓	1	2	3	4	5	6	Mean Rating	Cosine(i, 3) (user-user)	Pearson(i, 3) (user-user)
1	7	6	7	4	5	4	5.5	0.956	0.894
2	6	7	?	4	3	4	4.8	0.981	0.939
3	?	3	3	1	1	?	2	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2	0.645	-0.817

User-Based Collaborative Filtering

Example (Including Mean-Centred Ratings)

- Mean-Centred Ratings shown to the right.
- $\hat{r}_{31} = 2 + \dfrac{1.5 * 0.894 + 1.2 * 0.939}{0.894 + 0.939} \approx 3.35$
- $\hat{r}_{36} = 2 + \dfrac{-1.5 * 0.894 - 0.8 * 0.939}{0.894 + 0.939} \approx 0.86$
- Still agrees that item 1 should be prioritised over item 6, but the rating for item 6 is now lower than anything user 3 has previously rated.

Item-Id ⇒ User-Id ⇓	1	2	3	4	5	6
1	1.5	0.5	1.5	-1.5	-0.5	-1.5
2	1.2	2.2	?	-0.8	-1.8	-0.8
3	?	1	1	-1	-1	?
4	-1.5	-0.5	-0.5	0.5	0.5	1.5
5	-1	?	-1	0	1	1

User-Based Collaborative Filtering

Two Other Points…

- There are other similarity measurements (other than Pearson).

$$\text{Cosine Similarity} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

- There may be many more!
- Also, there are various prediction functions (more in the literature)
- Aforementioned prediction function:

$$\hat{r}_{uj} = \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot s_{vj}}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|} = \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot (r_{vj} - \mu_v)}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|}$$